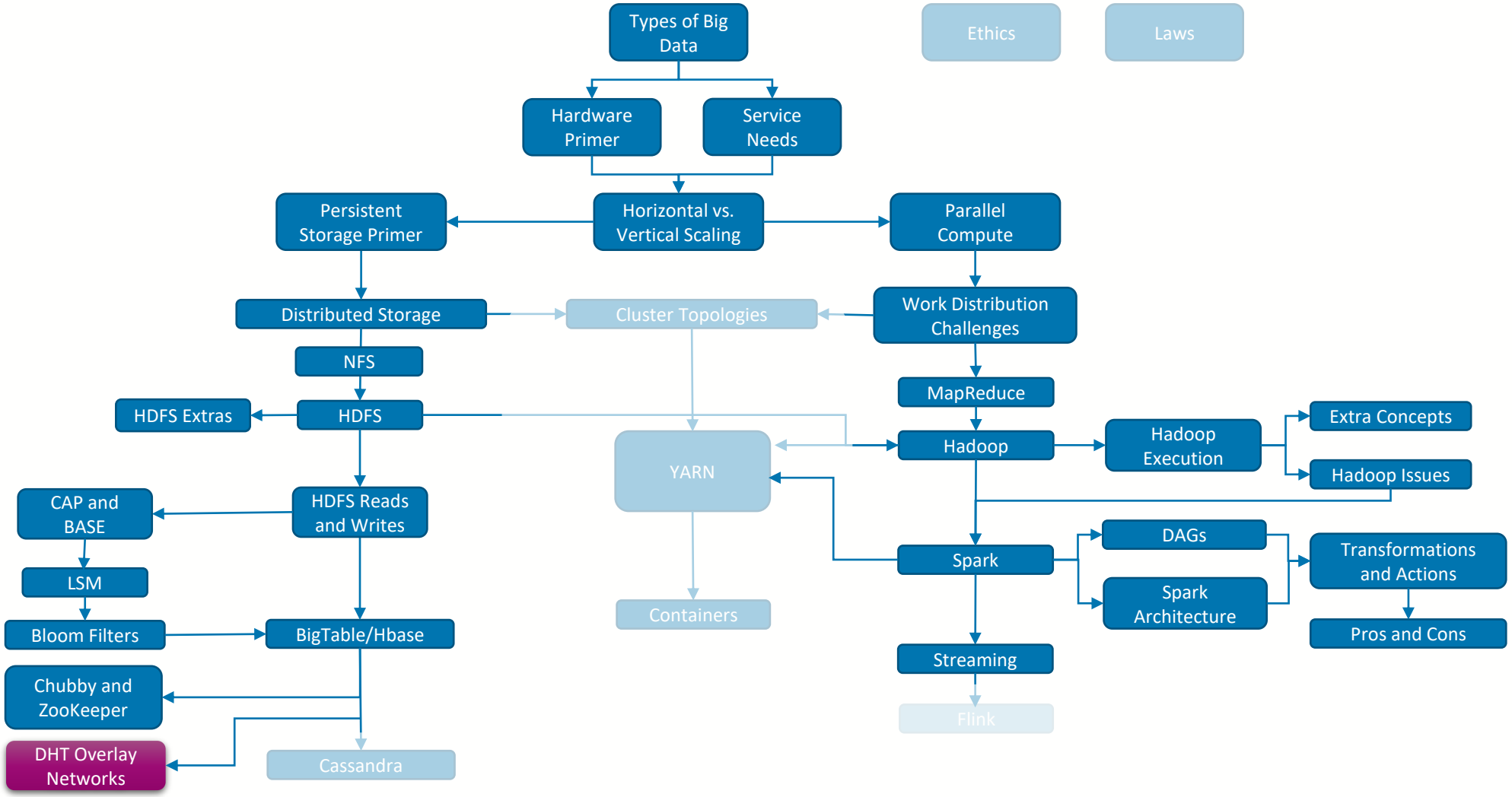
W8-04: DHT Overlay Networks

COMPSCI4064 (H) and COMPSCI5088 (M)

Dr. Richard McCreadie

University of Glasgow

Where are we?



University of Glasgow

The Problem with Masters

- Multiple times in these lectures I have pointed out where we have ‘master’ services and how these can be a single point of failure
  - Hadoop Job Tracker, Spark Master, NFS Server, HDFS Name Node, HBase HMaster

- As it turns out, if we are clever, we can build systems that don’t rely on a master service, there are just some additional costs involved
  - These are known as Peer-to-Peer services

University of Glasgow

Some Context with BitTorrent

- Probably the peer-to-peer data storage system you are most familiar with is BitTorrent
  - Allows users to share files across a network
  - Developed in 2001 by Bram Cohen from the University at Buffalo

- Core ideas
  - Break a large file into smaller pieces in a consistent way
  - When a peer wants a file, download many pieces simultaneously from other peers that currently hold the file
  - Coordinate who has what file via tracker servers
    - A downloader requests a list of available peers with pieces of a file in the form of a torrent file
    - The downloader reports their download state to tracker servers periodically

- The original version of BitTorrent still has a point of failure, which is the tracker servers

University of Glasgow

Reminder: What is an Index

- In a data storage system an index is simply a mapping between a filename (or other storage structure identifier like an HBase Region ID) and where the actual data for that object is stored (usually multiple locations for redundancy).
  - Example: In the HDFS, a <File Path,blockID> pair is mapped to one or more data nodes that holds that block
  - Example: In BitTorrent, a tracker file maps a hash for a piece of a file to one or more IP addresses of machines that (at some point) stored that file piece
- (an index may contain more than this, but this is the minimum we need to know)

Key	Value
/my/file/photo.png	21.241.21.11, 73.123.21.34
/my/file/readme.md	21.241.21.11, 73.123.21.34
/conf/setup.txt	9.4.52.123
/videos/movie.mp4	100.231.45.128, 198.186.34.1

University of Glasgow

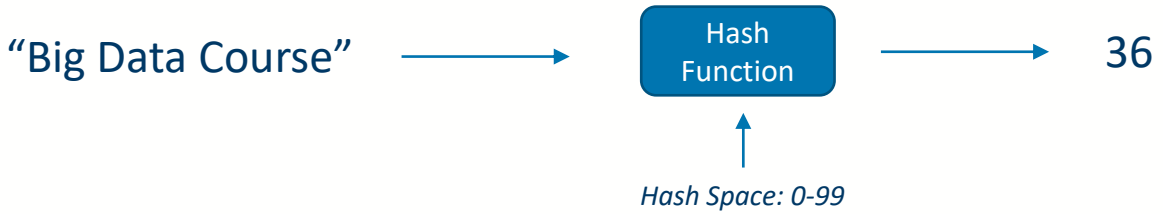
Hash-Based Peer Assignment

- Centralized tracker servers turned out to be an issue, but we need a way to find out who has what files
  - Let's distribute the file index amongst the peers as well!

- We want the information about a single file to be stored on a few (but not all) peers for redundancy
  - **Core Idea:** Don’t store the file metadata (e.g. its name) directly, instead use a hash

Hashing (You have seen this before)

- The process of transforming any given key or a string of characters into another (usually shorter) value.



- A Hash function takes as input a key/string and outputs a number or bit array representing that key/string
  - A hashing function will have a hash space, which defines the max-min values that can be output or the size of the bit array respectively
  - Given a input A the hash function will always give out value B
  - Multiple different inputs can result in the same output

Hash-Based Peer Assignment

- Centralized tracker servers turned out to be an issue, but we need a way to find out who has what files
  - So lets distribute the file index amongst the peers as well…

- We want the information about a single file to be stored on a few (but not all) peers for redundancy
  - **Core Idea:** Don't store the file metadata (e.g. its name) directly, instead use a hash
  - We can use the hash value to decide what peers will store the data about the file

University of Glasgow

Hash-Based Peer Assignment

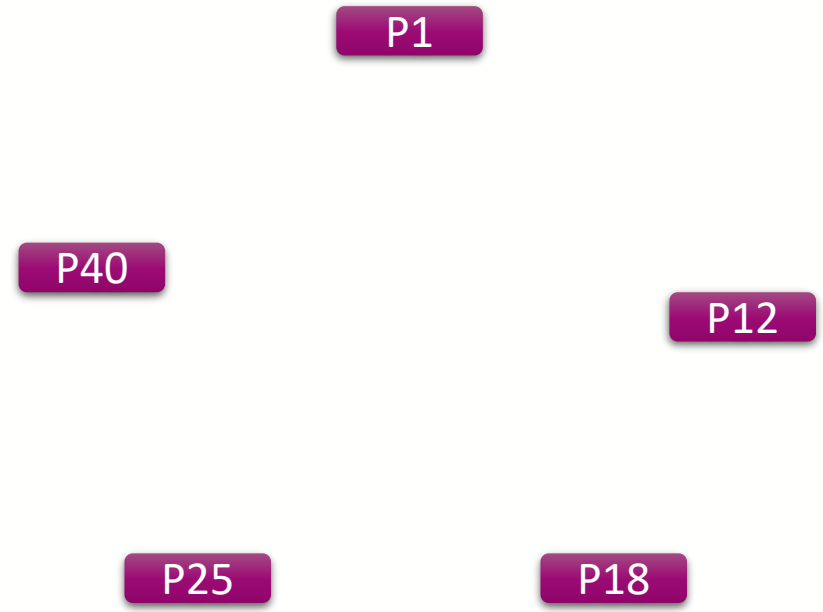
- Centralized tracker servers turned out to be an issue, but we need a way to find out who has what files
  - So lets distribute the file index amongst the peers as well…

- We want the information about a single file to be stored on a few (but not all) peers for redundancy
  - **Core Idea:** Don't store the file metadata (e.g. its name) directly, instead use a hash
  - We can use the hash value to decide what peers will store the data about the file
  - Use multiple Hash Functions to select multiple peers to hold the file metadata (enabling redundancy)

Peer ID Assignment

- When a peer joins the network, we assign it a peerID
  - This ID must exist in the same hash space as our file keys
    - No point having a peer with ID=100 if our files can only be allocated in a range 0-99.
  - Usually, the same hash function is used for both keys and peer identifiers
  - When a peer joins it requests all the index data associated to its peerID
  - When a node leaves it passes all its index data to the peer with the next highest ID

- This strategy of file assignment to peers based on a hashing function is known as Consistent Hashing
  - Given the file's key and the hash function we can always resolve the peer ID that was selected to hold the data about that file



Distributed Hash Table

- What I have just described also creates what is known as a Distributed Hash Table (DHT)
  - Different rows in this table are distributed amongst the peers
  - Note that we usually don't hash the file name, but rather a more unique representation of the file (e.g. an MD5 hash of the contents)

Key
/my/file/photo.png
/my/file/readme.md
/conf/setup.txt
/videos/movie.mp4

peerID (Hash)	Key	Value
36	/my/file/photo.png	21.241.21.11, 73.123.21.34
36	/my/file/readme.md	21.241.21.11, 73.123.21.34
12	/conf/setup.txt	9.4.52.123
40	/videos/movie.mp4	100.231.45.128, 198.186.34.1

Distributed Hash Table

University of Glasgow

Overlay Network

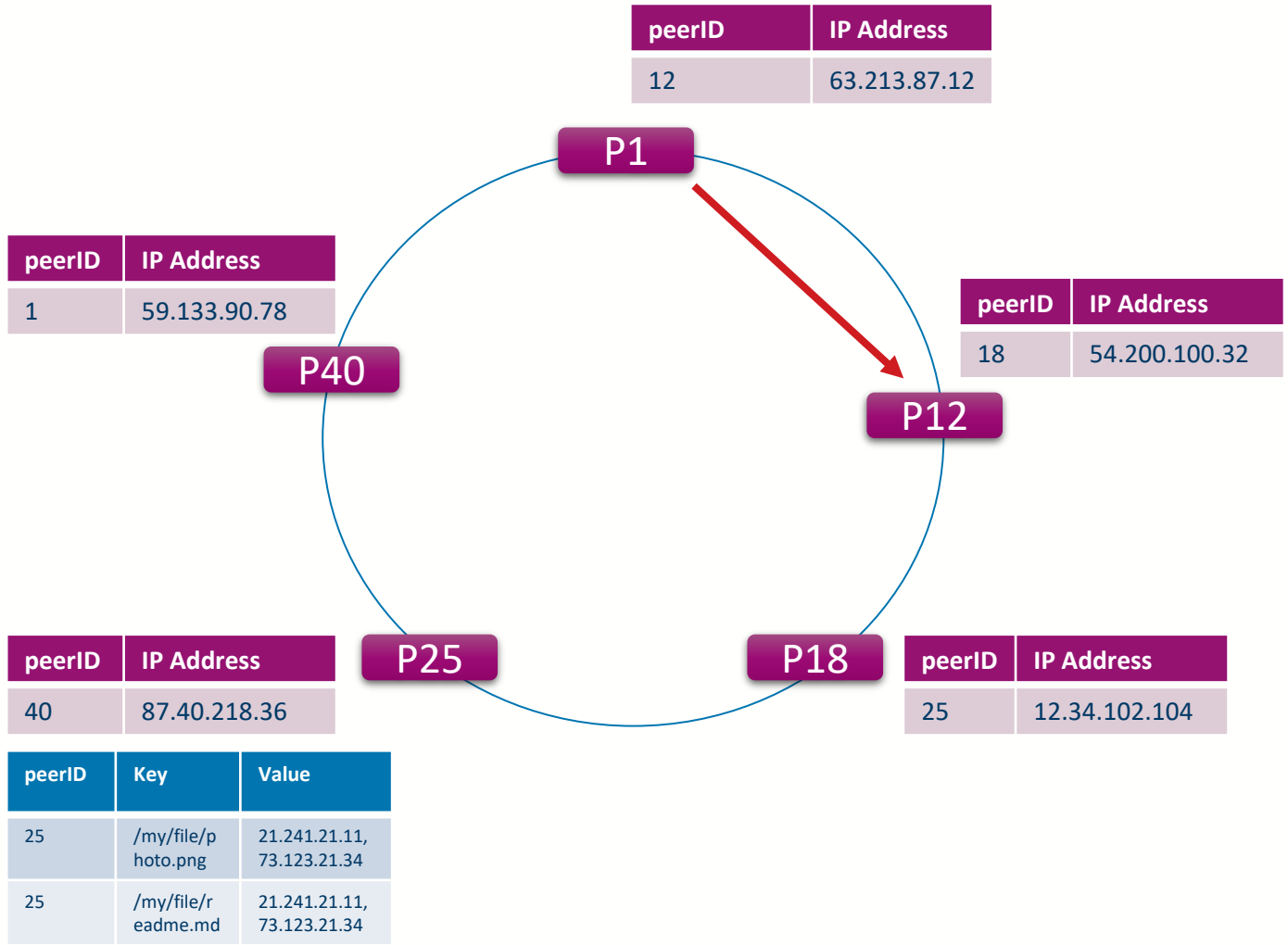
- Ok, so this hash table provides a mapping between a file and a peer's ID
  - But how does a node get from a peer's ID and its IP address?

- To solve this, we introduce the idea of an overlay network
  - This is simply the idea of having an additional protocol (set of rules) that allows us to find peers in the network

- What is the minimum information that a peer would need to know to translate from a peer's ID and its IP address?

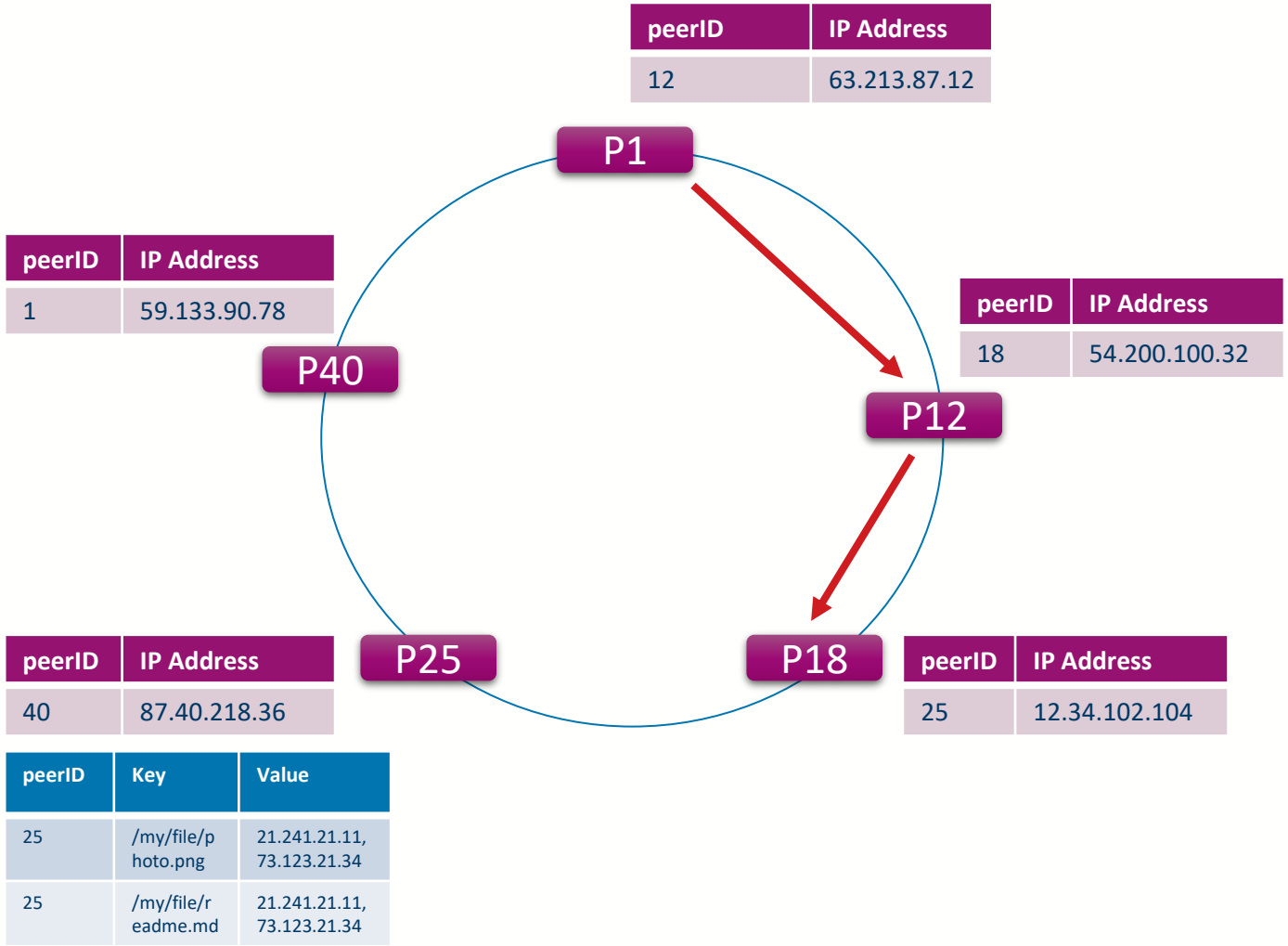
Simple Distributed Hash Table

- In the simplest overlay design, each peer would only need to know the peerID and IP address of the node with the next highest ID (its successor)
- We can pass a request around the network until we find the target peer
- For instance, if peer 1 wanted to get information from peer 25 it would
  - Ask peer 12



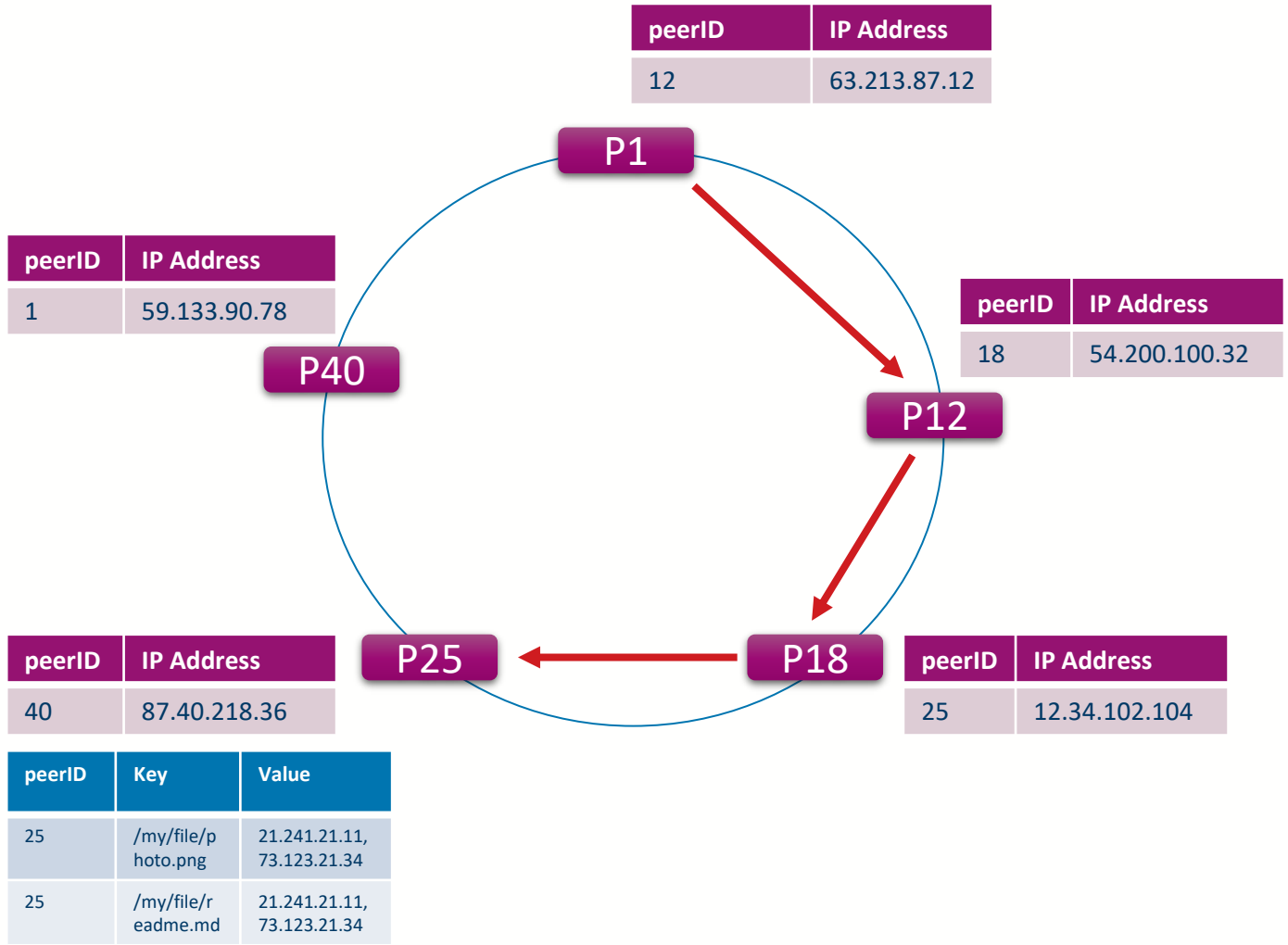
Simple Distributed Hash Table

- In the simplest overlay design, each peer would only need to know the peerID and IP address of the node with the next highest ID (its successor)
- We can pass a request around the network until we find the target peer
- For instance, if peer 1 wanted to get information from peer 25 it would
  - Ask peer 12
  - Which would ask Peer 18



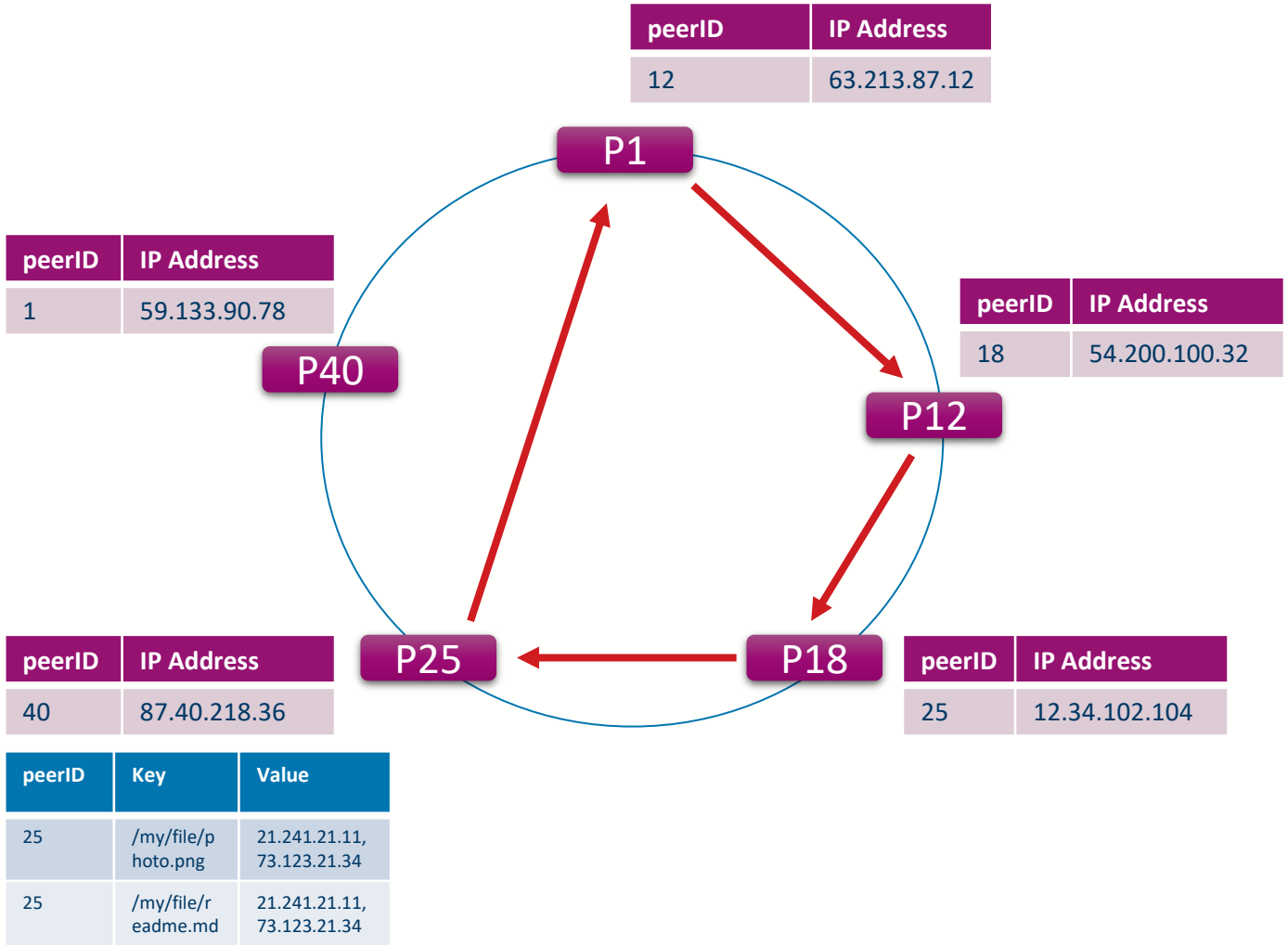
Simple Distributed Hash Table

- In the simplest overlay design, each peer would only need to know the peerID and IP address of the node with the next highest ID (its successor)
- We can pass a request around the network until we find the target peer
- For instance, if peer 1 wanted to get information from peer 25 it would
  - Ask peer 12
  - Which would ask Peer 18
  - Which would ask Peer 25



Simple Distributed Hash Table

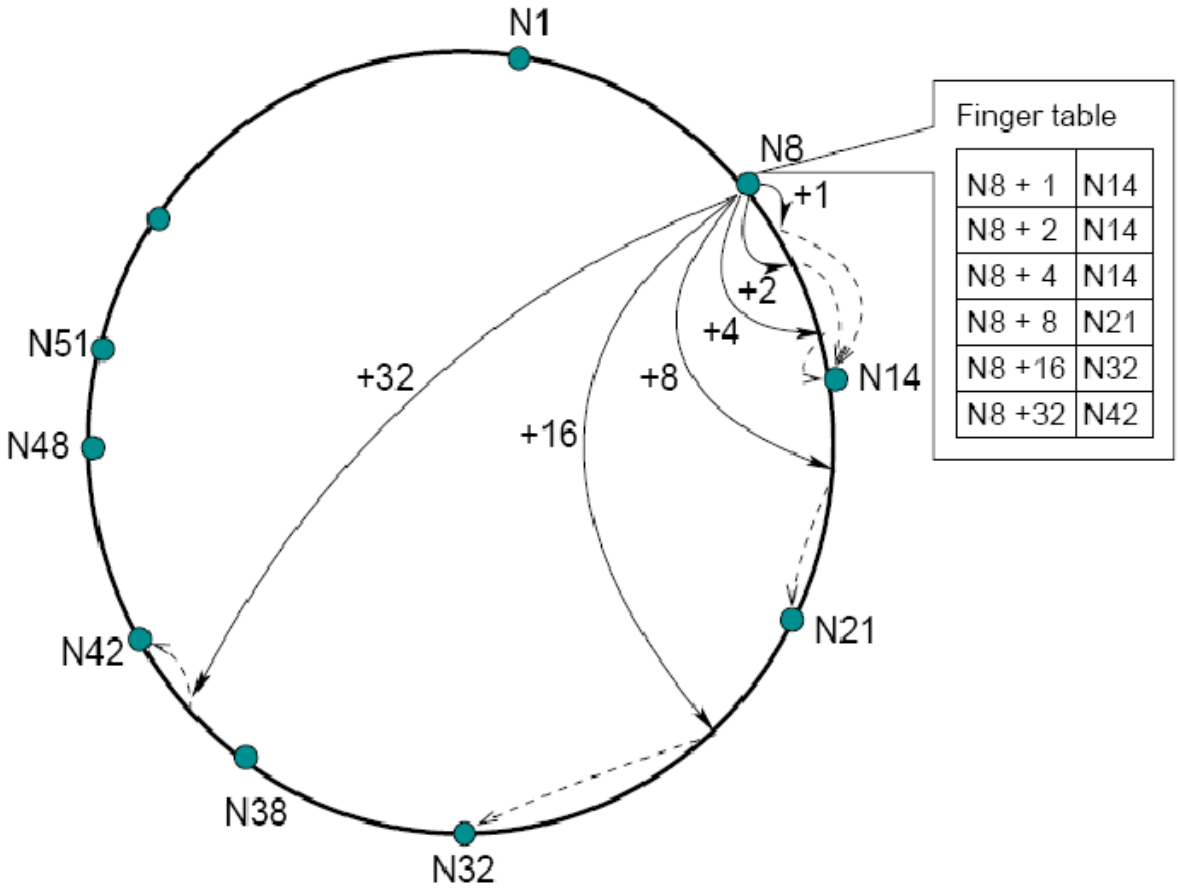
- In the simplest overlay design, each peer would only need to know the peerID and IP address of the node with the next highest ID (its successor)
- We can pass a request around the network until we find the target peer
- For instance, if peer 1 wanted to get information from peer 25 it would
  - Ask peer 12
  - Which would ask Peer 18
  - Which would ask Peer 25
  - Which would return the data to Peer 1



Finger Tables

- This is obviously very slow (O(n) hops)… how can we make this faster?

- Instead of only storing information about the successor, instead store information about m (=logN) other peers
  - The i_{th} row of the finger table of peer n points to the node responsible for ID $(ID(n) + 2^{i-1}) \bmod 2^m$
  - The first row points to the successor of n
  - Finger entries can point forwards and/or backwards (predecessors)



University of Glasgow

Finger Tables

- In an N-peer network, with high probability, the number of hops taken (nodes visited) during lookup() is: $O(log_2 N)$

- Intuition:
  - With m-bit IDs, the maximum distance that will be travelled is 2^m
  - At each step the search distance is halved
  - This is achieved without global knowledge, i.e. each node only has log_2N entries in its finger table

University of Glasgow

DHT Overlay Networks

- Allows us to build an index without a master and have no global knowledge on:
  - Which nodes are in the cluster
  - Which data items are stored in the cluster
  - What is the load of each node
  - Which are failed or non-responsive nodes
- The rows of the index are distributed amongst the peers in the network
- Peers in the network are assigned identifiers via a hash function (e.g. taking their IP address and port number as input)
- Assignment of index rows to peers is done via **Consistent Hashing**
  - Hash the key for each index entry to pick a peer to store the row
  - Multiple hash functions can be used to select multiple peers

Course Coordinator
Dr. Richard McCreadie

Email: richard.mccreadie@glasgow.ac.uk
Room: SAWB 304

#UofGWorldChangers
@UofGlasgow